Khmer OCR

Architecture & Process Diagrams

Visual step-by-step walkthrough of the entire research paper — from problem definition to final solution and results

1

CER on ID Cards

2

CER on Manuscripts

3

Training Images

4

Better than Tesseract

5

Languages Transfer

CNN + TRANSFORMER | RESNET-45 | 2D FEATURE MAPS | ADAPTIVE PATCHING | IMAGE CHUNKING

CROSS-LANGUAGE TRANSFER

Diagram 1 — The Big Picture

End-to-end journey from problem to state-of-the-art results

01

The Problem

Khmer script has stacked characters, tiny diacritics, no word spaces, and 5 vertical rows. Tesseract fails at 27–90% error rates.

02

Generate Synthetic Training Data

1.5M document images + 1.3M scene text images using 48 Khmer fonts, augmented with blur, noise, rotation, and perspective warp.

03

Design the Model Architecture

CNN (ResNet-45) extracts 2D features → Adaptive Patching preserves vertical info → Transformer Encoder → Transformer Decoder generates text.

04

Innovate: Image Chunking

Split long text images into overlapping chunks, process independently, merge predictions using confidence scores. 5× speed improvement.

05

Test on Real-World Data

Evaluate on Khmer ID cards, KHOB digital books, Sleuk Rith ancient manuscripts, and 5 other languages.

06

Results: State-of-the-Art

3.00% CER on ID cards (9× better than Tesseract), 2.46% on ancient manuscripts (60% better than previous best).

Diagram 2 — Why Khmer is Hard

A single syllable can occupy up to 5 vertical rows simultaneously — this is what makes legacy OCR fail

Row 1 — Superscript

◌ · Tiny diacritics (~15% height). Often invisible at low resolution.

Row 2 — Above-base Vowel

◌ី ◌ឿ ◌ើ · Above vowels (~25% height). Sit above the base consonant.

Row 3 — Base Consonant

ក គ ស ន · Main character (100% size). The anchor of every syllable.

Row 4 — Subscript Consonant (Coeng)

◌្ ◌្ ◌្ · Below consonants (~60% height). Can stack multiple levels deep.

Row 5 — Below-base Vowel

◌ុ ◌ូ · Below vowels (~25% height). Sit beneath the base consonant.

Example: The word ស្ត្រី (woman) uses 4 of 5 vertical rows simultaneously. A computer must identify ALL components and their relationships correctly to read it.

Diagram 3 — The Core Innovation

1D vs. 2D Feature Maps — why preserving vertical information is the key breakthrough

❌ Old Method — 1D Feature Sequence

CRNN / TrOCR approach

> Input: H=8 rows × W=64 cols × C=512 channels

> Operation: COLLAPSE height (average/pool)

> Output: 1 row × W=64 × C=512

> ⚠️ Row 1 diacritics = Row 4 subscripts — ALL vertical info LOST

CER: ~27% (lost vertical structure)

✅ New Method — 2D Feature Map with Patches

This paper's approach

> Input: H=8 rows × W=64 cols × C=512 channels

> Operation: PATCH (keep all rows per column)

> Output: 64 patches, each [8 × 512] = 4096-dim

> ✓ Row 1 info preserved · ✓ Row 4 info preserved · ✓ All intact

CER: ~3% (vertical structure preserved)

Diagram 4 — Model Architecture (Stages 0–2)

Raw image → CNN backbone → Adaptive 2D Patching with exact tensor dimensions

Stage 0 — Input Image

Cropped text-line image resized to fixed height of 64px (preserving aspect ratio).

Shape: H=64 × W=variable × C=3 (RGB)

Why 64px? 32px loses tiny diacritics (12.1% CER) — 64px captures all 5 rows clearly (3.1% CER). Above 64px = no gain.

Stage 1 — CNN Backbone (ResNet-45)

Convolutional filters extract visual features at multiple abstraction levels:

64×512×3 → Conv1: edges/corners → 32×256×64 → Conv2: strokes → 16×128×128

Conv3: char parts → 8×64×256 → Conv4: full shapes → Output: H=8 × W=64 × C=512

Height 8 rows still preserves all 5 Khmer vertical zones — critical for diacritics!

Stage 2 — Adaptive 2D Patching Module

Input: 8×64×512 → Divide into column-wise patches (each = one column across ALL 8 rows).

Each patch: [8×1×512] → Flatten: [8×512] = 4096-dim vector → Linear projection → 512-dim.

Add positional encoding. Output: 64 patch vectors × 512-dim.

Unlike 1D: each patch has SEPARATE positions for Row 1 diacritics, Row 3 base, Row 4 subscripts!

Diagram 4 — Model Architecture (Stages 3–5)

Transformer Encoder → Decoder auto-regressive generation → Unicode text output

Stage 3 — Transformer Encoder (12 Layers)

Processes all 64 patches simultaneously with self-attention — each patch communicates with every other.
Architecture: 12 layers · 8 attention heads · Model dim 512 · Feed-forward dim 2048 · Dropout 0.1
Each layer: Multi-Head Self-Attention → Feed-Forward Network → Layer Norm + Residual Connection
Output: 64 enriched patch vectors × 512-dim

Stage 4 — Transformer Decoder (Auto-regressive)

Step 0: Input: [START] → Predict: ស

Step 1: Input: [START, ស] → Predict: ◌្ (coeng)

Step 2: Input: [START, ស, ◌្] → Predict: ត

Step 3: Input: [START, ស, ◌្, ត, ◌ុ] → Predict: រ

Step 4: Input: [START, ស, ◌្, ត, ◌ុ, រ] → Predict: ◌ី

Step 5: Input: [START, ស, ◌្, ត, ◌ុ, រ, ◌ី] → Predict: [END] → Final: ស្ត្រី ✓

Vocabulary: ~120 tokens (114 Khmer Unicode codepoints + [PAD] [UNK] [START] [END]). Stage 5 — Final Output: Khmer Unicode Text — can now be stored, searched, translated, or spoken aloud.

Diagram 5 — Image Chunking & Merging Algorithm

How very long text lines are split, processed, and recombined — with 5× speed gain

01

Original Long Image (64px × 2000px)

Full Khmer text line: ជំរាបសួរ អ្នកទាំងអស់គ្នា រស់នៅសុខសប្បាយទេ នេះជាប្រយោគវែងមួយ — too long for direct processing.

02

Split into Overlapping Chunks

Divide into 5 chunks of W=400px each, with 100px overlap between neighbors. Overlap prevents cutting characters at boundaries. Chunks: [ជំរាបសួរ អ្នក] [អ្នកទាំងអស់គ្នា] [គ្នា រស់នៅសុខ] [សុខសប្បាយទេ នេះ] [នេះជាប្រយោគវែង]

03

Process Each Chunk Independently

Each chunk runs through the full CNN + Transformer pipeline. Confidence scores: Chunk 1: 0.97 · Chunk 2: 0.95 · Chunk 3: 0.96 · Chunk 4: 0.94 · Chunk 5: 0.93

04

Merge Predictions by Confidence

Use center-of-chunk text, discard overlapping edges. Final output: ✓ ជំរាបសួរ អ្នកទាំងអស់គ្នា រស់នៅសុខសប្បាយទេ នេះជាប្រយោគវែងមួយ

4M

Attention Operations

Without chunking — $O(2000^2)$

800K

Attention Operations

With 5 chunks — $O(5\times400^2)$

5×

Speed Improvement

Faster processing

Diagram 6 — Training Data Generation Pipeline

How 2.8 million synthetic training images were created from scratch

Text Sources

Khmer Wikipedia, news articles, Khmer Bible, and literary texts — diverse real-world Khmer language content.

48 Fonts

Various Khmer fonts covering different styles — from modern digital to traditional print.

Rendering

Text rendered as images at various sizes and layouts, simulating real document and scene conditions.

Augmentation

Blur, noise, rotation, perspective warp, compression — making the model robust to real-world degradation.

Document Text Dataset

1.5M

Clean text on white/light backgrounds. Simulates scanned documents, PDFs, printed pages.

Augmentations: Gaussian blur, salt-and-pepper noise, JPEG compression, brightness variation, slight rotation.

Scene Text Dataset

1.3M

Text on real-world photo backgrounds. Simulates signs, billboards, menus, storefronts.

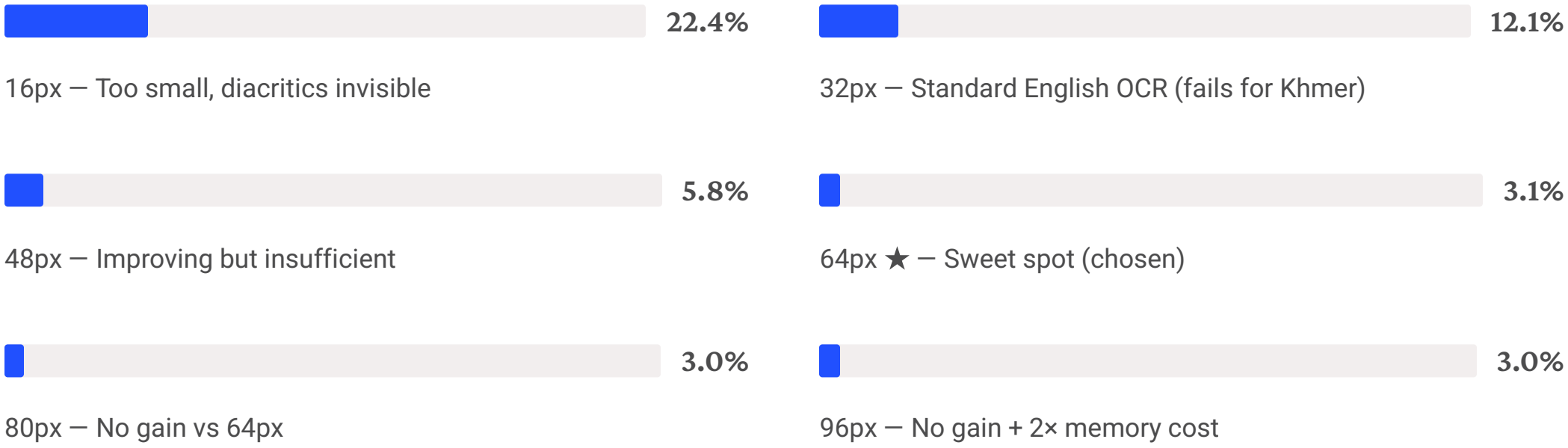
Augmentations: Perspective warp, elastic deformation, motion blur, shadow effects, color jitter.

2.8M Total Training Images

Split: Train 80% / Validation 10% / Test 10% · Fully annotated at character level · 48 Khmer fonts

Diagram 7 — Impact of Image Height on Accuracy

Why the standard 32px used in English OCR fails catastrophically for Khmer

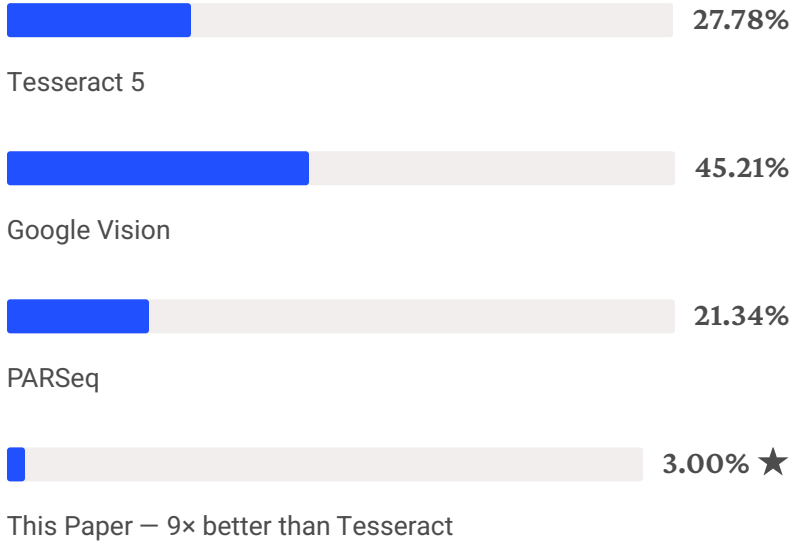


> Sweet Spot = 64px — Going from 32→64px cuts error by 74%. Going beyond 64px gives ZERO improvement but doubles memory cost. This explains much of why previous Khmer OCR systems failed — they used 32px, simply too short for a script with 5 vertical rows.

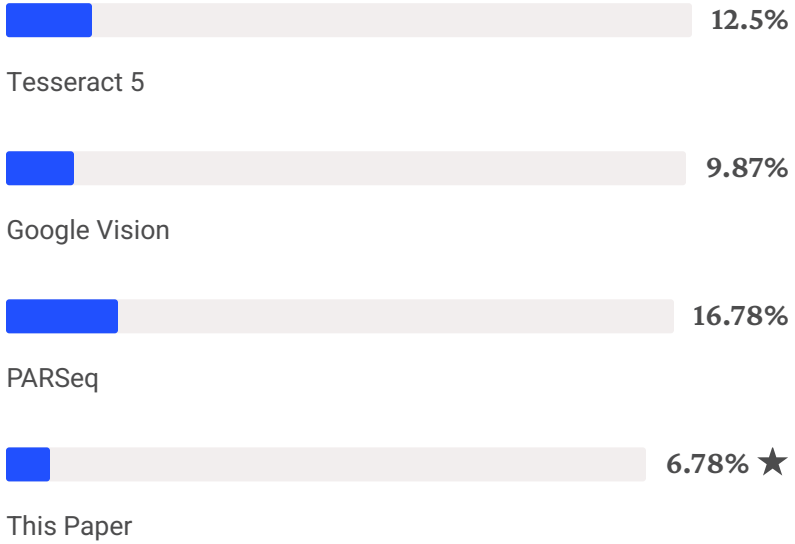
Diagram 8 — Results Dashboard

Performance on all real-world Khmer test sets. Lower CER = better.

Khmer ID Card — Name Field



Khmer ID Card — Address Field



KHOB Digital Books

> Key Result: 3.00% CER on ID cards — 9× better than Tesseract (27.78%). 2.46% CER on ancient manuscripts — 60% better than previous best.

Diagram 9 — Chunking Impact: CER vs. Text Line Length

Without chunking, errors explode on long lines. With chunking, errors stay flat.

1–10 chars

Without chunking: 2.8% · With chunking: 2.7% · Minimal difference on short lines.

11–30 chars

Without chunking: 3.1% · With chunking: 3.0% · Still minimal difference.

31–60 chars

Without chunking: 5.9% · With chunking: 3.2% · 1.8× better with chunking.

61–100 chars

Without chunking: 9.4% · With chunking: 3.5% · 2.7× better with chunking.

100+ chars

Without chunking: 14.7% · With chunking: 4.1% · 3.6× better — critical for real documents.

5×

Error increase without chunking (2.8% → 14.7%)

1.5×

Error increase with chunking (2.7% → 4.1%)

3.6×

Max improvement on 100+ char lines

> Without chunking: error explodes from 2.8% → 14.7% as lines get longer (5× worse). With chunking: stays stable 2.7% → 4.1% (only 1.5× increase). Essential for processing real documents!

Diagram 10 — Cross-Language Transfer Learning

Khmer pre-trained model fine-tuned on 5 other scripts with only 100K images each

Pre-trained on 2.8M Khmer images → Fine-tuned with only 100K images per language

Lao — −55% CER Reduction

7.89% → 3.56% · Most similar to Khmer (no spaces, diacritics)

Thai — −50% CER Reduction

8.43% → 4.21% · Stacked characters, similar structure

Burmese — −46% CER Reduction

9.12% → 4.88% · Subscript consonants

Hindi — −41% CER Reduction

6.78% → 4.02% · Diacritics above/below base

Vietnamese — −27% CER Reduction

5.34% → 3.89% · Latin + diacritics (most different from Khmer)

> Pattern: More similar to Khmer → greater improvement. Lao (closest relative): −55% | Vietnamese (Latin-based): −27%. This proves Khmer's challenges are a superset of other scripts — solving Khmer generalizes broadly.

Diagram 11 — Known Failure Cases

Where the system still struggles — and why

Low Resolution

Images below 40px tall: tiny diacritics become invisible. The difference between ṅ and ṅ (a single dot) disappears at low quality.

Text Detector Errors

If the upstream text detector crops too tightly, Row 1 diacritics above the text get cut off. The recognizer never sees them.

Ambiguous Characters

Some pairs like ◌ं vs ◌ँ differ by just one stroke length. At low quality, even humans struggle to distinguish them.

Rare Triple Clusters

Triple-stacked consonants (base + 2 subscripts) are rare in training data. The model sometimes drops the deepest subscript.

Historical Script Variation

Ancient palm-leaf manuscripts use character forms very different from modern fonts. Without fine-tuning, error rates jump significantly.

Summary — The Complete Journey

From problem to state-of-the-art Khmer OCR solution

The Problem

Khmer OCR fails at 27–90% CER. Stacked characters, 5 vertical rows, tiny diacritics — no existing tool could handle it.

2.8M Training Images

1.5M document + 1.3M scene text images. 48 Khmer fonts with heavy augmentation.

2D Feature Maps

Preserve vertical diacritic positions. Column-wise adaptive patching — the core innovation.

Image Chunking

Stable CER on long lines. 5× faster processing via overlapping chunk merging.

Results

3.00% CER on ID cards — 9× better than Tesseract. 2.46% CER on ancient manuscripts.

Cross-Language Transfer

5 languages: Lao −55%, Thai −50%, Burmese −46%, Hindi −41%, Vietnamese −27%.

> By solving Khmer OCR — one of the hardest non-Latin scripts — the researchers created a generalizable baseline that transfers to Thai, Lao, Burmese, Vietnamese, and Hindi with minimal additional data. The two key innovations (2D feature maps + image chunking) are applicable to any script with vertical structure.

ResNet-45 backbone · 64px image height · 12-layer Transformer encoder · ~120-token vocabulary · Column-wise adaptive 2D patching · Overlapping chunk merging by confidence score